

Proyecto Final Simulación de Procesos Distribuidos usando Docker, gRPC y Relojes Vectoriales.

Descripción General

Desarrollar una simulación de un sistema distribuido compuesto por varios procesos independientes, donde cada proceso estará implementado dentro de un contenedor Docker.

Cada proceso deberá ejecutar su propio servidor RPC usando gRPC, enviar mensajes a otros procesos mediante un cliente RPC, mantener un reloj vectorial para registrar el orden lógico de los eventos, y registrar eventos internos, envíos y recepciones en una bitácora.

El objetivo del proyecto es comprender cómo se mantiene el orden lógico de eventos en sistemas distribuidos mediante el uso de relojes vectoriales.

El sistema estará compuesto por 5 contenedores Docker. Cada contenedor representará un proceso distribuido e incluirá:

- Servidor gRPC
- Cliente gRPC
- Reloj vectorial propio
- Registro de eventos (bitácora)

Servicios RPC

Los servicios **gRPC a Implementar son**

1. Envío de mensajes a otros procesos con parámetros: id del que envía, , id del que recibe , mensaje, reloj vectorial

Aquí se incrementa el reloj vectorial del emisor, se envía el mensaje junto con el vector, se registra evento de envío.

2. Recepción de mensajes desde otros procesos, con parámetros : id del que envía, mensaje, vector vectorial recibido.

Aquí se actualiza el reloj vectorial, se registra el evento de recepción y se devuelve un ACK.

3. Simula un evento interno en el proceso. No tendrá parámetros y aquí se incrementa el reloj vectorial local y se registra evento interno.
4. Envía un mensaje a todos los procesos del sistema. Esto para simular la difusión de mensajes, comunicación distribuida, actualización lógica simultánea

Bitacoras

Respecto a las bitácoras, cada proceso deberá generar un registro de eventos como se indica en el siguiente ejemplo, considerar que el inicio de la línea es el evento.

Ejemplo

[INTERNAL] P2 vector=[0,3,0,0,0]

[SEND] P1 -> P3 msg="hola" vector=[2,0,0,0,0]

[RECEIVE] P3 <- P1 msg="hola" vector_recibido=[2,0,0,0,0] vector_actualizado=[2,0,1,0,0]

Escenario de Ejecución

Plantear un escenario para el sistema compuesto por 5 procesos. Cada proceso tendrá una secuencia de eventos, tanto internos como de comunicación. Un ejemplo es el siguiente, pero deben hacer un diagrama como loe hechos en clase para relojes vectoriales. Además los eventos internos deben ser propuestos.

Proceso P1

1. Evento interno: calcular suma del 1 al 100
2. Envía mensaje a P3
3. Evento interno: generar número aleatorio

Proceso P2

1. Evento interno: cálculo de factorial(6)
2. Envía mensaje a P5
3. Recibe mensaje de P4

Proceso P3

1. Recibe mensaje de P1
2. Evento interno: multiplicar matrices 2x2
3. Envía mensaje a P4

Proceso P4

1. Evento interno: contar del 1 al 500
2. Envía mensaje a P2
3. Recibe mensaje de P3

Proceso P5

1. Recibe mensaje de P2
2. Evento interno: promedio de números aleatorios
3. Broadcast a todos los procesos

Requerimientos

Todo el sistema debe ejecutarse dentro de contenedores Docker. Los contenedores deben ejecutarse simultáneamente, la comunicación debe realizarse únicamente entre contenedores y cada proceso debe mantener su propio reloj vectorial.

Se debe implementar de forma correcta el Reloj Vectorial

Cada evento debe quedar registrado.

Las pruebas deberán realizarse utilizando ***docker exec***

Entregables.

Los archivos y un documento donde se incluya lo siguiente:

1. Archivo IDL .proto con la definición de los servicios.
2. Servidor RPC completo.
3. Cliente RPC
4. Implementación del Reloj de Lamport.
5. Diagrama donde se defina un escenario concreto para el sistema compuesto por 5 procesos: Cada proceso tendrá una secuencia de eventos, tanto internos como de comunicación.
6. Descripción de orden de instrucciones para poner en marcha los contenedores
7. Bitácoras de ejecución.
8. Video corto de demostración donde se muestre ejecución de contenedores, envío de mensajes, actualización de relojes vectoriales, bitácoras

Criterios de Evaluación

Criterio	Porcentaje
Diseño de escenario	10%
Comunicación entre procesos. Esto involucra el correcto funcionamiento de la definición de servicios , , el Servidor RPC y Cliente RPC	20%
Implementación correcta del reloj vectorial Cada evento debe quedar registrado.	20%
Funcionamiento en Docker Descripción de orden de instrucciones para poner en marcha los contenedores. Las pruebas deberán realizarse utilizando <i>docker exec</i>	20%
Bitácoras y registro de eventos Cada evento debe quedar registrado.	15%
Demostración y explicación en video	15%